

Simplifying the AI Agent Loop with Function Calling

Function Calling and the Simplified Agent Loop

Now that we have explored function calling, we can significantly simplify our AI agent loop. Traditionally, parsing the LLM's responses required engineering at each output format, validating them, and handling errors when the model deviated from the requested structure. With function calling, the model now naturally suggests structured responses, ensuring we receive either a valid function call or a standard text reply. This drastically reduces the complexity of parsing responses and handling errors, instead focusing on ensuring responses flow consistently from one validated state, we can simply request the function call provided by the model at various times with confidence.

How Function Calling Simplifies the Agent Loop

Let's revisit the agent loop, but now using function calling instead of handling multiple types of parsing, so that the model's LLM function suggested function calls when needed.

```
import os
from typing import List

# The function to use for weather
def get_weather(location: str) -> str:
    """Fetches the weather for a given location"""
    # Here, a file is returned
    # Note: This is a simplified version of the function
    # In a real-world scenario, you would use an API
    # and return the weather data as a string
    # For example: "The weather in New York is sunny."
    # Note: This is a simplified version of the function
    # In a real-world scenario, you would use an API
    # and return the weather data as a string
    # For example: "The weather in New York is sunny."
    return "The weather in " + location + " is sunny."

def main():
    """The main function that runs the agent loop and provides a summary message"""
    # Create a new OpenAI client
    client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

    # Create a new chat completion
    chat_completion = client.chat.completions.create(
        messages=[{"role": "user", "content": "What's the weather in New York?"}],
        model="gpt-4o",
    )

    # Extract the function call from the chat completion
    function_call = chat_completion.choices[0].message.function_call

    # If there is a function call, use it to get the weather
    if function_call:
        # Create a new function call object
        function_call_obj = FunctionCall(
            name=function_call.name,
            arguments=function_call.arguments,
        )

        # Call the function
        result = get_weather(function_call_obj.arguments["location"])

        # Add the result to the chat completion
        chat_completion.choices[0].message.function_call_result = result

    # Print the chat completion
    print(chat_completion)

    # Print the function call result
    print(function_call_obj.result)
```

What This Changes

- 1. **No More Custom Parsing Logic:** We don't need to engineer logic that instead parsing the model always returns structured JSON for function calls.
- 2. **Simpler Response Handling:** Instead of a tight loop that constantly checks what the agent should do, we simply wait and receive the function call.
- 3. **Unified Text & Action Handling:** The function call is either the model request with a strongly defined structured and action state workflow.
- 4. **Automated Function Resolution:** The agent generally expects the tool name from the model to be corresponding Python function and executes it with the provided arguments.

Errors Can Still Occur

While function calling improves structured responses, it does not eliminate all potential issues. One common problem is that the model may still sometimes return improperly formatted JSON or omit call that isn't valid. For example, the response could contain syntax errors, missing required fields, or incorrectly formatted data. If the agent attempts to parse the malformed JSON, it will fail. To address this, we need to add error handling around `[json.loads()]` to catch `[json.JSONDecodeError]`. At parsing error occurs, the agent detects by sending another request. This prevents the error loop from escalating and ensures robustness in execution. You can do this as a practice exercise to enhance the agent's stability. Additionally, even when the JSON is correctly formatted, errors can still arise at runtime due to missing files, incorrect arguments, or unexpected edge cases in function execution. Our error handling ensures that these issues do not halt execution entirely but instead provide meaningful feedback for debugging and improvement.

Key Takeaways

By leveraging function calling, we reduce complexity from the agent loop, allowing the AI to interact with the automated world more stably. This simplification makes AI agents more robust, scalable and easier to integrate into real-world applications.

Our agent design can still require time to **navigate back dynamically with decisions**, further improving flexibility and adaptability to the agent design.